RobiOmni ROS2 Chassis Package User Guide

1 Tested Environment

Operating system: Ubuntu 22.04.5 LTS
ROS distribution: ROS2 Humble
Package version: robiomni V1.0.2
Workspace archive: robiomni_ros2.zip

For additional details, see README.md in the ROS package.

2 USB Permissions for the CAN Adapter

Method 1: Create a udev rule to grant access.

Run the following command:

```
sudo bash -c "echo 'ACTION==\"add\",SUBSYSTEMS==\"usb\", ATTRS{idVendor}==\"04d8\", ATTRS{idProduct}==\"0053\", GROUP=\"users\", MODE=\"0777\"' >> /etc/udev/rules.d/99-usbcan.rules"
```

After running the command, reconnect the USB cable of the CAN adapter or restart the system for the rule to take effect.

Method 2: Set permissions manually.

Run the following command:

```
lsusb
```

Run lsusb and note the Bus ID and Device ID for the CAN adapter listed as Microchip Technology, Inc.

Run the following command:

```
sudo chmod 777 /dev/bus/usb/<BUS_ID>/<DEVICE_ID>
```

Replace <BUS_ID> and <DEVICE_ID> with the values recorded above to grant access to the device.
Example: sudo chmod 777 /dev/bus/usb/001/006
After changing permissions, no reconnection or restart is required.

3 Create the ROS Workspace

Extract robiomni_ros2.zip into your home directory to create ~/robi_ros2. For a manually created workspace, copy both robiomni and robot_ros2_msgs into src.

```
mkdir -p ~/robi_ros2/src
cd ~/robi_ros2
```

4 Build the ROS Package

Run these commands from the workspace root with ROS2 Humble installed. If rosdep has not been initialized, first run sudo rosdep init and rosdep update.

Run the following command:

```
cd ~/robi_ros2
source /opt/ros/humble/setup.bash
rosdep install --from-paths src --ignore-src -r -y --rosdistro humble
colcon build --packages-up-to robiomni --executor sequential
```

5 Load the Environment

Run the following command:

```
source /opt/ros/humble/setup.bash
source ~/robi_ros2/install/setup.bash
```

6 Start the ROS Package

Connect the CAN adapter, then run:

Run the following command:

```
ros2 launch robiomni robiomni.launch.py
```

7 Monitor Published Topics

After starting the ROS package, use the commands below to read published topic data.

Read raw CAN data:

```
ros2 topic echo /robiomni/frame_info
```

Read front-left steering drive faults:

```
ros2 topic echo /robiomni/drive_steer_front_left_error_info
```

Read front-right steering drive faults:

```
ros2 topic echo /robiomni/drive_steer_front_right_error_info
```

Read rear-left steering drive faults:

```
ros2 topic echo /robiomni/drive_steer_rear_left_error_info
```

Read rear-right steering drive faults:

```
ros2 topic echo /robiomni/drive_steer_rear_right_error_info
```

Read front-left drive faults:

```
ros2 topic echo /robiomni/drive_front_left_error_info
```

Read front-right drive faults:

```
ros2 topic echo /robiomni/drive_front_right_error_info
```

Read rear-left drive faults:

```
ros2 topic echo /robiomni/drive_rear_left_error_info
```

Read rear-right drive faults:

```
ros2 topic echo /robiomni/drive_rear_right_error_info
```

Read automatic charging status:

```
ros2 topic echo /robiomni/auto_charge_info
```

Read battery information acquired by the main controller:

```
ros2 topic echo /robiomni/battery_info
```

Read chassis status:

```
ros2 topic echo /robiomni/state_info
```

Read remote control data:

```
ros2 topic echo /robiomni/remote_control_info
```

Read chassis steering motor angles:

```
ros2 topic echo /robiomni/angle_info
```

Read chassis motor currents:

```
ros2 topic echo /robiomni/current_info
```

Read drive motor encoder counts:

```
ros2 topic echo /robiomni/encoder_count_info
```

Read chassis motion status:

```
ros2 topic echo /robiomni/motion_info
```

Read chassis motor speeds:

```
ros2 topic echo /robiomni/speed_info
```

Read chassis odometry:

```
ros2 topic echo /robiomni/odom_info
```

8 Control the Chassis with Topics

After starting the ROS package, publish commands to the following topics to control the chassis.

Linear velocity, angular velocity, and steering angle control:

```
ros2 topic pub -r 50 /robiomni/velocity_ctrl geometry_msgs/msg/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Omnidirectional FTFD: straight travel at 0.1 m/s. Set linear.x=0.1 and angular.z=0.

```
ros2 topic pub -r 50 /robiomni/ftfd_ctrl geometry_msgs/msg/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Turn while moving: linear.x=0.1 m/s and angular.z=0.1 rad/s. angular.z is angular velocity in this mode; linear.y controls lateral velocity in m/s.

```
ros2 topic pub -r 50 /robiomni/ftfd_ctrl geometry_msgs/msg/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0.1}}'
```

Stop this mode: stop the previous publisher, then continuously publish zero values on the

same topic. Confirm the chassis has stopped before pressing Ctrl+C.

```
ros2 topic pub -r 50 /robiomni/ftfd_ctrl geometry_msgs/msg/Twist '{"linear":{"x":0,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Ackermann: straight travel at 0.1 m/s. Set linear.x=0.1 and angular.z=0.

```
ros2 topic pub -r 50 /robiomni/ackermann_ctrl geometry_msgs/msg/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Turn while moving: linear.x=0.1 m/s and angular.z=0.1 rad (about 5.7 degrees). angular.z is the steering angle, not angular velocity; keep linear.y=0. This is not rotation in place.

```
ros2 topic pub -r 50 /robiomni/ackermann_ctrl geometry_msgs/msg/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0.1}}'
```

Stop this mode: stop the previous publisher, then continuously publish zero values on the same topic. Confirm the chassis has stopped before pressing Ctrl+C.

```
ros2 topic pub -r 50 /robiomni/ackermann_ctrl geometry_msgs/msg/Twist '{"linear":{"x":0,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Double Ackermann: straight travel at 0.1 m/s. Set linear.x=0.1 and angular.z=0.

```
ros2 topic pub -r 50 /robiomni/double_ackermann_ctrl geometry_msgs/msg/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Turn while moving: linear.x=0.1 m/s and angular.z=0.1 rad (about 5.7 degrees). angular.z is the steering angle, not angular velocity; keep linear.y=0. This is not rotation in place.

```
ros2 topic pub -r 50 /robiomni/double_ackermann_ctrl geometry_msgs/msg/Twist '{"linear":{"x":0.1,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0.1}}'
```

Stop this mode: stop the previous publisher, then continuously publish zero values on the same topic. Confirm the chassis has stopped before pressing Ctrl+C.

```
ros2 topic pub -r 50 /robiomni/double_ackermann_ctrl geometry_msgs/msg/Twist '{"linear":{"x":0,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}'
```

Rotation in place: use this separate topic with linear.x=linear.y=0 and angular.z=0.1 rad/s. Here angular.z is angular velocity; this is separate from the Ackermann steering-angle command.

```
ros2 topic pub -r 50 /robiomni/rotate_ctrl geometry_msgs/msg/Twist '{"linear":{"x":0,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0.1}}'
```

Chassis motion control:

```
ros2 topic pub -r 50 /robiomni/motion_ctrl robot_ros2_msgs/msg/FourWheelSteerMotion '{"motion_mode":1,"linear":0,"steering":0,"angular":0,       "linear_x":0.1,"linear_y":0,"angular_z":0}'
```

Independent wheel-speed control is not implemented in this package version. Use the supported chassis motion control interfaces above.

Clear collision status:

```
ros2 topic pub /robiomni/collision_clean std_msgs/msg/Empty '{}' -1
```

Disable automatic charging:

```
ros2 topic pub /robiomni/auto_charge_ctrl std_msgs/msg/UInt8 '{"data":0}' -1
```

Enable infrared automatic charging:

```
ros2 topic pub /robiomni/auto_charge_ctrl std_msgs/msg/UInt8 '{"data":1}' -1
```

Turn lighting off:

```
ros2 topic pub /robiomni/lamp_ctrl std_msgs/msg/Bool '{"data":false}' -1
```

Turn lighting on:

```
ros2 topic pub /robiomni/lamp_ctrl std_msgs/msg/Bool '{"data":true}' -1
```

Reset odometry:

```
ros2 topic pub /robiomni/odom_clean std_msgs/msg/Empty '{}' -1
```